

The VHDL Code Standard

- [About the Code Standard](#)
- [Application note](#)
- [New at work or working here as a consultant](#)
- [File Structure and Format](#)
- [RTL Design Rules](#)
 - [Clock Domain Crossing \(CDC\)](#)
 - [Naming and conventions](#)
- [Module Test](#)
- [Verification and Sign-off](#)
- [Appendix 1: Type Conversions](#)
- [Appendix 2: Sigasi Linting](#)

About the Code Standard

Authors	Fredrik Kristensen, Kim Steenberg, Anders Bering and Kenneth Haldbæk.
Last edited by	Fredrik Kristensen (2026-01-20)
Version	1.2

Application note

This is the code standard that should be used to the extent possible. There will be times when, due to circumstance beyond your control, it will be impossible to meet the standard or when meeting the standard will reduce the quality of the code. In those cases break the rule, but please comment it with name, date and reason. **Use common sense.**

New at work or working here as a consultant

Please, send you first module for review to FKRI. Don't spend too much time implementing before you send it to review. The sooner we can align your code-standard to our, the better.

File Structure and Format

Module:

- A module contains the following sub-folders:
 - hdl_src
 - vhdl source files.
 - hdl_tb
 - testbench files

- test_case
 - test case setup and wave-file.
 - syn (optional)
 - setup of Quartus build.
- A module must have a file that points out every package and sub-module used by this module. Located in the hdl_src folder.
 - Today manip-files are used.

File naming:

- The module is named mod_name
- The main file in the hdl_src folder must be named mod_name.vhd
- The main file in the hdl_tb folder must be named mod_name_tb.vhd
- If a vhdl source file is named vhdl_src.vhd has a file package then this file must be named vhdl_src_p.vhd and located in the hdl_src folder
- If a vhdl source file is named vhdl_src.vhd then the following files must exist in the folder with the naming shown below
 - hdl_tb folder vhdl_src_tb.vhd
 - scripts folder run_alint_vhdl_src.py
 - scripts folder run_cov_vhdl_src.py
 - scripts folder run_sim_vhdl_src.py
 - test_case folder config_vhdl_src.cfg
 - test_case folder sim_wave_vhdl_src.tcl

File:

- A VHDL file and the entity it contains must have the same name.
 - One entity + architecture per file.
- Every entity must have a testbench.
 - Unless completely tested as sub-module in another module.
- Files must start with a header, see example below.

Register interfaces:

- Shall be specified in xml-format according to Register-tool standard.
 - Register constants for vhdl usage, shall be generated through Register-tool.
 - Register constants for SW usage, shall be generated through Register-tool.
 - Register documentation shall be generated through Register-tool.

Code:

- Write enough comments
 - Comments shall be written in spell-checked English.
 - Use line comments, avoid block comments.
- Lines should never be longer than 256 characters.
- Use sections for easier reading, see examples below.

- All HDL code is written in lower case.
- Only one assignment or one declaration per line.

Section example:

```
-----
-- Constants
-----

-----
-- Types
-----

-----
-- Signals
-----
```

VHDL Header:

All vhdL-files must have a header, and it shall look like this:

```
-----
-----
-- Copyright 202X Everllence, Tegllholmsgade 41, 2450 Copenhagen SV, Denmark
-----
-----
--
-- Requirements: 1. Requirements specific for this module in priority order.
--               2. Can also be a pointer to other documentation, if too
complex to write here.
--               3. There can be modules without specific requirements. Then
leave this field empty.
--               4. That the module works as expected and can handle the
project specific clock-frequency
--               are implicit requirements and don't need to be stated
here.
--
-- Description:  Module description.
--
-- Revision log:  Date:          Initial:  JIRA:
--               202x-xx-xx  OLLE:      [TRIT-XXXX] JIRA heading + file
specific comment if meaningful *Note 1
--
-----
-----
```

***NOTE 1:** The revision log shall as far as possible only contain one line per JIRA-case, i.e. **NOT** one line per commit. There could however be cases where an update to an old JIRA-case is done after other JIRA-cases have been added to the revision log, in this case it's OK to add the same old JIRA-case one more time in the revision log, with a comment on why it was done.

VHDL Header example:

```

-----
-- Copyright 2023 Everllence, Tegllholmegade 41, 2450 Copenhagen SV, Denmark
-----
--
-- Requirements:      1. Correctly interface ADC: Texas Instruments
ADS8363/ADS7263/ADS7223.
--                    - According to datasheet SDAS523C Oct 2010 revised
jan 2017.
--                    2. Compensate for board signal delays (primarily from
isolators).
--                    - According to timing requirements in Doc :
triton_func/boards/com/sdc/ TB updated(fkri,2023-03-01)
--                    3. Provide data samplerate as 1050 ns *
gc_no_of_channels, i.e. 2100, 3150 or 4200 ns.
--                    - HW filters are designed accordingly.
--                    - FPGA controlled filters settings are based on these
values (PS/DO).
--                    - SW controlled filters settings are based on these
values (AI/CIOM).
--
-- Description:      This module is intended for the adsxxx3 series of ADCs.
They are used on
--                  the DI_DO_1430 and DI_DO_1560 Boards for measuring
currents in the DO outputs
--                  the AI_AO 1440/1/2 Board for measuring the 8 currents in
the AI channels,
--                  the PS_1390 and the CIOM_1610/2 Board for measuring the 8
currents in the AI channels,
--
--                  The ADC is running in MODE I with both M0 and M1 set to
'0', the Half-Clock Mode
--                  is used for conversion and reading data from the ADC, the
muxes are used in the
--                  Pseudo Differential 4:1 configuration (PDE = '1').
--                  Output data channels are gc_no_of_channels x 2. Where
gc_no_of_channels is 2, 3 or 4.
--
--
-- Revision log:      Date:          Initial:    JIRA:
--                  2023-01-30    FKRI          [TRIT-2305] New common ADC-interface
to adsxxx3-ADCs
--                  2023-02-23    ORZI          [TRIT-2379] Reset for clock 200 MHz
added.
--
-----

```

RTL Design Rules

These rules are to the extent possible checked by the linting rules.

- **IEEE Libraries. Use the following:**
 - use ieee.std_logic_1164.all;

- use ieee.numeric_std.all;
- **Include Packages:**
 - Preferable use full path, e.g.:
 - use work.pk_avalon_st;
 - manio_rx_d2s $\leftarrow$ pk_avalon_st.c_avalonst_d2s_idle;
 - Local package can use .all, e.g. in module tachometer :
 - use work.pk_tachometer_defs.all; – register-tool generated package
 - use work.pk_tachometer_const.all; – Own module package
- **VHDL version:**
 - New language features can be used if supported by simulation, linting and synthesizing tools.
- **Naming conventions, as shown in Table 1:**
 - Clock signal is clk_i
 - Reset signal is reset_i
- **Entity ports:**
 - Use only modes in and out with name suffixes _i or _o.
 - Mode inout is allowed preferable in FPGA top-level if possible.
 - FPGA top-level entity port should contain a FPGA-family generic to allow modules to include correct IPs depending on FPGA-family.
 - gc_fpga_family : string; – FPGA type e.g. “Arria V” or “Cyclone V”
 - Types std_logic and std_logic_vector
 - Or records of these.
 - Or arrays of these.
 - Registered outputs. If not possible mark reason.
- **Components:**
 - Direct instantiation for RTL code is recommended.
 - Must be instantiated with named signal mapping, i.e. do not use ordered mapping.
- **IPs:**
 - Use vendor IPs if possible, they are most likely better tested and more used, than anything new and homemade.
 - If possible, use our standard modules, e.g., synchronizer, filters, and CRC modules. These modules are found in the ip_lib-submodule.
- **RAMs and FIFOs**
 - Always use the generic RAM and FIFO found in the IP_lib.
 - Some FIFOs and RAMs will be instantiated using a vendor IP.
 - If a module that use a vendor specific IP, is used in multiple platforms, make sure that it is simulated using all vendor types.
 - This is easiest done by creating a simulation configuration files for each vendor.
- **Synchronous processes:**
 - Always sensitive to clock only.
 - Clock event is always on rising edge.
 - Reset is synchronous.
 - All control registers must be initialized in reset.
 - Preferably one if/case-state per process.
- **Combinatorial processes:**

- Sensitivity list includes all signals that are read in the process.
 - VHDL2008 use (ALL).
 - Complete if-clauses must be used.
 - Signals are assigned in every branch unless a default value is used.
 - Only one if/case-state per process.
- **Finite State Machine (FSM):**
 - Preferably use 1-process FSMs with all assignments based on state done in the FSM.
 - Easier to read for other designers.
 - Other-state:
 - Don't use other-state for functionality
 - Always include an other-state as an "error-state", with a return to a known state and an assert.
- **Gating:**
 - Use data gating not clock gating.
- **Design for testability:**
 - No latches.
 - No combinational feed-back loops.
 - No data and clock coupling, i.e. don't use clock signal as data signal.
 - No redundant or dead-code.
- **Clock Domain Crossing (CDC):**
 - See below chapter on [CDC](#).
- **Signal priority:**
 - Clear has priority over set, if architecture doesn't require otherwise.
- **Signals:**
 - No hard-coded signal values, use constants or generics instead.
 - Indexes of `std_logic_vector` are defined as downto.
 - Integers shall be defined with a range.
 - All bit assignments shall be marked with radix, i.e. B"" or X"".
 - Limit arrays to max 2-dimensions.
- **Generics:**
 - Always range-constraint generics.
 - Use asserts to check valid generic combinations, if not ALL combinations are supported and verified.
 - Avoid to use default values of generics when instantiating modules.
 - To increase visibility of generic settings.
 - Allowed exception: if an old module is retro-fitted with a new generic, default value can be used to avoid updating all instances of this module.
- **Constants:**
 - The register-tool provides top-level configuration constants and all register-interface constants.
- **Variables:**
 - Use variables to show intermediate step in complex assignments.
- **Records**
 - Use records to collect groups of signals.
- **Functions**

- Allowed if required, but don't over do it.
- **Procedures**
 - Not allowed, unless cleared with FKRI.

Clock Domain Crossing (CDC)

Clock domain crossing is a complex topic and differs from case to case. In **Figure 1**, the most common ways to do CDC is shown. For every CDC case do:

- Perform CDC on an as high level as possible in the design, preferably in the top-level.
 - This is to avoid routing and use of unsynchronized signals in sub-modules.
- Don't mix complex functionality with CDC.
 - Contain CDC in it's own section or module.
- If a module contains signals from more than one clock domain, clearly indicate which signals belongs to which clock domain.
 - prefix all signals belonging to the non-standard clock domain with <clock-domain name>_<signal-name>, for example:
 - serdes_rx_clk_i : in std_logic; -- Serdes RX clock
 - serdes_rx_data_i : in std_logic; -- data_i from serdes_rx clock domain
- Call for a review of the CDC.
- Check build tools for warnings regarding CDC.
- Always use the synchronizer module in IP_lib
 - It contains the correct constraints.
- Always use the generic FIFO and RAM module in IP_lib.
- One synchronizer component per signal.
 - Never concatenate signals into the same synchronizer, this will add unnecessary constraints.
- Always add a comment section about the performed CDC, stating:
 - Fast and slow side
 - expected data transfers, e.g. burst, slow changing, fast changing.
- More information about the technical issues with CDC can be found in:
 - [Clock domain crossing \(CDC\) - The complete reference guide](#)
 - [Clock Domain Crossing \(CDC\) Design & Verification Techniques Using SystemVerilog from SNUG 2008 by Clifford E. Cummings](#)

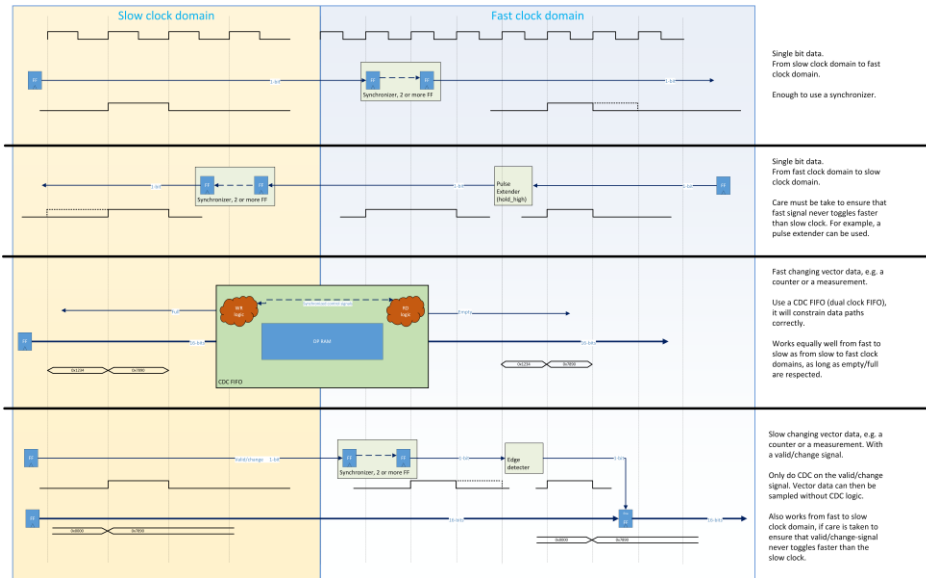


Figure 1, common CDC implementations.

Naming and conventions

Item	Prefixed	Postfixed
Type	t_	
State	s_	
Procedure	pr_	
Constant	c_	
Variable	v_	
Generic	gc_	
Process	p_	
Function	f_	
Alias	al_ (note 1)	
Instantiation/component	u_	
Generate	g_	
Package	pk_	
Signal in non-common clock domain	<clock-domain name>_ (note 3)	
Array of vectors/records		_a (note 2)
Active low signal		_n (note 2)

Entity input		<code>_i</code>
Entity output		<code>_o</code>
Entity inout		<code>_io</code>

NOTE 1: alias only allowed in testbench.

NOTE 2: place before in/out postfix, e.g. `tx_s2d_a_i` or `chip_select_n_o`.

NOTE 3: prefix all signals belonging to the non-standard clock domain with `<clock-domain name>_<signal-name>`, for example:

`serdes_rx_clk_i` : in `std_logic`; -- Serdes RX clock

`serdes_rx_data_i` : in `std_logic`; -- `data_i` from `serdes_rx` clock domain

Module Test

Testbench:

- Architecture name must be `tb`
- Black-box testing is required,
 - Black-box testing means:
 - Change inputs and test outputs.
 - Don't use internal states of the design when testing. The same TB can then be reused if the implementation is changed.
- Written in VHDL.
 - Test IPs may be in Verilog if required.
- Test case order:
 1. Reset and out of reset tests
 2. Normal usage test.
 1. This test shows other designers how this module is intended to be used.
 2. Stimuli data shall be normal expected input data.
 3. All other tests.
- Use random values in your test-bench, e.g.
 - Random delay between input stimuli.
 - Random data stimuli.
 - Random errors.
 - Random test-sequence.
- Initialize the random value generator with a random seed:
 - variable `RV : RandomPType`;
 - `RV.InitSeed(random_seed)`;
 - Once the test-bench is running on Jenkin, the '`random_seed`' is changed every day.
- All test-benches must include the following packages:
 - OSSVM packages
 - `use work.RandomBasePkg.all ;`
 - `use work.RandomPkg.all ;`
 - `use work.CoveragePkg.all;`

- use work.AlertLogPkg.all;
 - use work.MessagePkg.all;
 - use work.TranscriptPkg;
 - use work.common_tb_pkg.all;
 - use work.common_register_interface_pkg.all;
 - Only if module has a register interface.
- Follow the RTL Design rules if feasible.
 - Use variables, functions and procedures freely.
- Aliases are allowed.
 - Prefix aliases with al_.
- Print overview messages in the log, to the extent that someone else can get a rudimental overview of which tests are performed.
 - If all tests passed, the last message must be "Test Pass"

If module:

- Is part of a data chain:
 - Include if possible data source module in simulation.
 - Include if possible data sink module in simulation.
- Interface a FPGA-external component:
 - Include a simulation model of the external component and the interconnect.
 - Use vendor models if available.
 - Implement the simulation model as a component for easy reuse, i.e. make it a MAN-TEST-IP.
- Includes standard interfaces, e.g. register or streaming-data interfaces:
 - Use TEST-IPs to verify these interfaces. These modules are found in the sim_lib-submodule.
- Contains data calculations:
 - Make a reference module.

Verification and Sign-off

All code must pass the following criteria before sign-off:

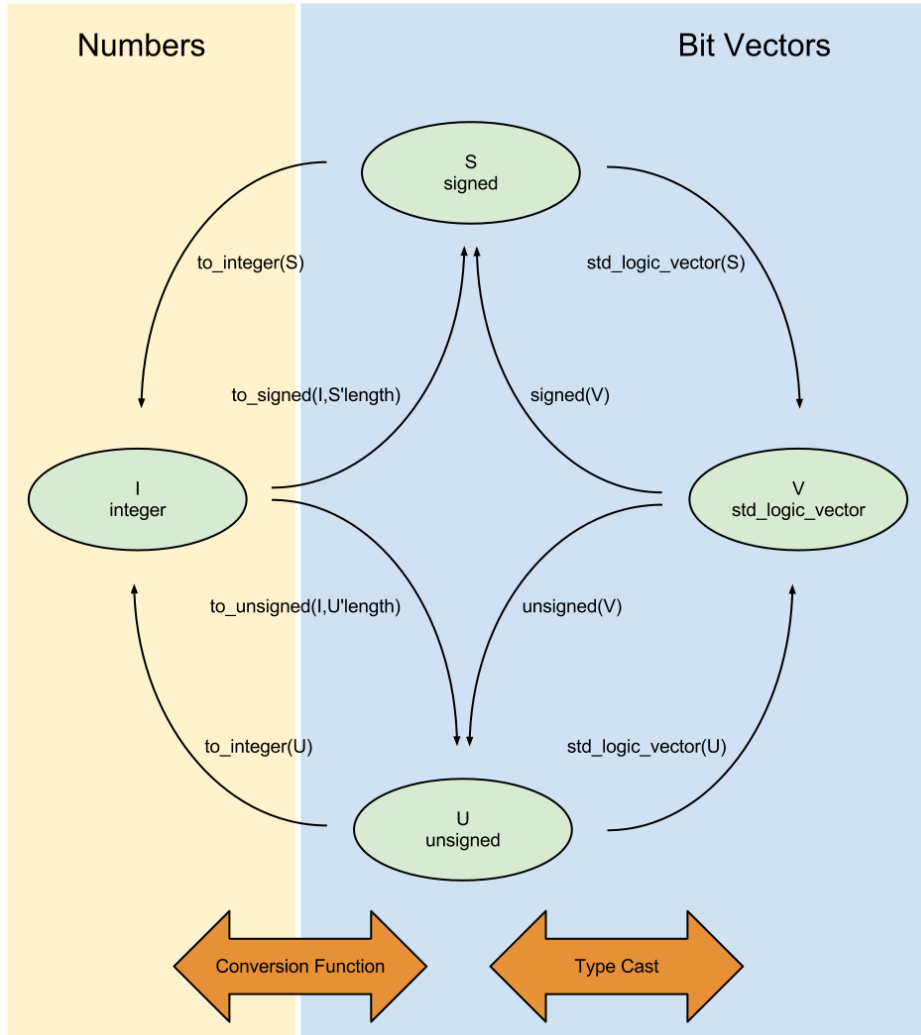
- **Functional verification.**
 - Self-checking tests.
 - End with a “Test Pass” message.
- **Code coverage:**
 - **100% line coverage**
 - Exceptions must be marked with initials, date and reason.
 - **100% branch coverage**
 - Exceptions must be marked with initials, date and reason.
 - **100% toggle coverage if possible**
 - The way to reach 100% toggle-coverage is to:
 - 1) Write all your testcases.
 - Make sure that line and branch coverage is 100%

- 2) Check your toggle-coverage.
 - Can it be improved with additional tests?
 - Can it be improved by (within reason) increasing run time of your tests?
 - 3) Still not 100 % toggle-coverage:
 - Disable toggle-coverage in the test configuration.
 - **NOTE: Never use alias to force toggle internal signals in the module in order to get higher toggle coverage!**
- **Linting rule check**
 - Every violated (bypassed) rule must be marked with initials, date and reason.
- **Code inspection**
 - As a minimum do a compare to the old-code before you commit the code.
 - Beyond compare is handy for this
 - Request a visual inspection by other designers if critical/new code.
- **Code is cleaned:**
 - no unused signals, variables or constants.
 - VSC (Sigasi-plugin) is handy for this.
 - All out-commented code is removed.
 - Header is up-to-date.
 - No TODOs in the code:
 - All TODOs should be removed and replaced with a JIRA.
- **No undefined signals** after asserted reset and rising edge on clock.
 - Does not apply to uninitialized memories and the like.

Before sign-off it's highly recommended that:

- Code is beautified, e.g. code is lined up on “:” and “←” .
 - VSC (Sigasi-pluin) is handy for this.
- Simulation wave file is up-to-date.
- The module is built stand-alone at slightly higher clock frequency than intended.
 - Use project assigned tool, e.g. Quartus or Xilinx.
- The module is incorporated in the current projects top-level system simulation and a sanity check of this module is added in the top-level simulation.
- Check all comments for validity.

Appendix 1: Type Conversions



Appendix 2: Sigasi Linting

In Sigasi most of the code standard can be linted for while writing the code.

For this the attached files can be used. They must be placed in the .settings in the root of the directory with the project.

Minimum supported Sigasi version is 5.4.



com.sigasi.hdt.vhdl.Vhdl.prefs



com.sigasi.hdt.vhdl.linting.prefs